

每个 Chunk 一秒钟:VLA 推理预处理管线的 GPU 化与缓存优化方案

问题测量、成本结构分析与分阶段实施计划 (LingBot-VLA 2.0 / lerobot PR #3967)

LingBot-VLA 加速工作组 测试平台:NVIDIA A100-SXM4-80GB 状态: 待评审实施

摘要 对 LingBot-VLA 2.0 部署链路的实测显示: 模型推理本身经优化后已降至约 400 ms, 而 CPU 预处理管线在多相机配置下高达 ~1 s, 是模型耗时的 2.6 倍, 且完全未被官方 130 ms 的延迟声明覆盖 (该口径只计 sample_actions)。预处理随每次推理调用执行一次 (每个动作 chunk 一次, 而非每个控制步一次), 在异步部署模式下, 端到端延迟必须小于 chunk 覆盖的墙钟时间——当前 1.4 s 的端到端延迟已击穿 25 步 @ 30 Hz 的 833 ms 预算。本方案提出三项纯工程改造: ①图像管线整体 GPU 化 (uint8 原图直接上卡,resize/归一化/patchify 用融合算子完成); ② episode 级 tokenizer 缓存 (task 字符串在 episode 内不变, 每 chunk 重复 tokenize 纯属浪费); ③预处理与去噪推理的跨 chunk 流水。预期端到端延迟进入 500 ms 以内, 使异步模式下的高频控制成立。全部改动为纯 PyTorch/工程手段, 不引入专用内核。

关键词: 数据预处理;GPU 卸载;tokenizer 缓存;流水线重叠;VLA 部署

1 问题: 被漏计的 2/3 延迟

官方 upstream(robbyant/lingbot-vla-v2)README 声明 “4090D 上单次推理约 130 ms(10 步去噪)”, 但该数字来自 deploy.lingbot_vla_v2_policy 中仅包裹 sample_actions 的计时打印, 预处理完全在口径之外。我们在 A100 上对官方 deploy 路径做了全链路分段实测 (robotwin 三相机 480×640, 真实 6B 权重,bf16 + compile 最快配置):

表 1: 官方 deploy 全链路分段 (最快配置; 宿主机有 CPU 负载, 绝对值偏高但结构成立)

环节	耗时 (ms)	占比	执行位置
预处理 (resize + 图像处理器 + tokenizer + 特征变换)	1023.6	71.0%	CPU(numpy/PIL)
模型推理 sample_actions(含 compile)	391.2	27.1%	GPU
动作反归一化 unapply	1.2	0.1%	CPU
端到端 infer 合计	1442.5	100%	—

模型侧持续优化 (dense MoE、compile) 已把推理压到 ~400 ms, 预处理随之成为唯一的大头——这正是提出本方案的直接动机。

2 执行频率与预算约束

预处理不是每个控制步都跑, 而是随每次推理、每个动作 chunk 跑一次:chunk 执行模式下,step 0 新观测 → 预处理 → 推理 → 50 步 chunk;step 1...24 直接执行 chunk, 不推理不预处理;step 25

chunk 耗尽, 重新进入下一轮。预处理每 chunk 一次, 被 use_length(25–50) 个控制步摊薄。

是否需要优化, 取决于部署模式的硬约束。异步重叠模式 (执行当前 chunk 的同时计算下一个) 要求:

$$T_{e2e} = T_{\text{preprocess}} + T_{\text{model}} \leq \text{use_length} / f_{\text{ctrl}} \quad (1)$$

表 2: 预算核算 (三相机配置实测值代入)

部署参数	chunk 墙钟预算	当前 e2e	判定
25 步 @ 30 Hz	833 ms	1442 ms	超支 73%
50 步 @ 30 Hz	1667 ms	1442 ms	勉强, 余量 13%
同步阻塞模式	—	1442 ms	每次 chunk 停顿 1.4 s, 不可用

结论: 在真机高频控制下, 预处理优化不是锦上添花, 而是异步模式成立的前提。仿真批量评测场景下它只影响总时长, 优先级可降低。

3 成本结构: 为什么 CPU 管线慢

对官方 `_prepare_model_input` → `FeatureTransform.apply` → `prepare_images` 链路的代码审查给出预处理的三个成本来源:

- **全数组遍历 × 单线程带宽。**每路相机从 uint8 原图到 patch 张量要经历约 6 次完整数组遍历: 类型转换、torchvision Resize(256×256)、Qwen3-VL image processor 的 smart_resize、rescale、normalize、pad/patchify。每次遍历都新分配数 MB 的 numpy 数组, 三相机再乘 3, 外层还有 per-camera 的 Python 循环。
- **数据传输方向颠倒。**现状是“CPU 处理完 → fp32 大张量最后才 `.to(device)`”; 正确方向是“uint8 原图 (3 路共 2.8 MB) 直接 pinned + 异步 H2D(<0.5 ms) → 处理全程在卡上”。
- **与推理的 launch 线程抢 CPU。**该模型推理是 launch-bound(GPU 忙时仅 ~225 ms/次), CPU 时间极其宝贵; 预处理独占 CPU ~1 s, 直接拖慢两侧的 kernel 发射——宿主机有负载时预处理从 ~300 ms 恶化到 1023 ms 即为此机制。

tokenizer 一项是次要成本: task 字符串整个 episode 不变, 却每 chunk 重新渲染 chat template(含数百个 image_pad 占位符)、重新编码, 单次约数 ms——收益小但实现几乎免费。

4 方案设计

4.1 图像管线 GPU 化 (主收益项)

将每相机的“转换 → resize → 归一化 → patchify”从 numpy/PIL 改为卡上融合算子, 并把多相机从 Python 循环改为单次批量调用:

Listing 1: 图像管线 GPU 化示意

```
# 现状 (每相机, CPU): uint8 -> float32 -> Resize -> smart_resize -> rescale -> normalize
#                               -> patchify
# 方案 (所有相机, GPU):
imgs = torch.from_numpy(np.stack(frames))           # [Ncam, H, W, 3] uint8
imgs = imgs.pin_memory().to("cuda", non_blocking=True)
x = imgs.permute(0, 3, 1, 2).float()               # 融合进后续算子
```

```
x = F.interpolate(x, size=(Hs, Ws), mode="bilinear", align_corners=False) # ==
    smart_resize
x = (x * rescale - mean) / std # 广播逐点, 可与 patchify 融合
patches = rearrange(x, ...) # temporal/spatial merge 即
    reshape/permute
```

- 数学上与 HF processor 逐步对齐 (双线性 resize 参数、均值方差、padding 规则), 以单元测试锁定逐元素等价;
- 输出直接位于 GPU, 下游 vision tower 零拷贝消费;
- 多相机 stack 成 batch 维, 一次 kernel 覆盖全部视角。

4.2 episode 级 tokenizer 缓存

在 deploy 包装层以 (task 字符串, 图像 token 数) 为键缓存 lang_tokens 与 lang_masks:episode 首次推理时计算, 后续 chunk 直接复用。键含 image_token_count 是因为 Qwen3-VL 的 chat template 会把图像占位符展开进 prompt——图像分辨率不变时展开结果亦不变。预期收益数 ms 级, 实现成本极低, 顺手完成。

4.3 跨 chunk 流水 (部署侧, 可选)

异步模式下, 下一观测的图像预处理与当前 chunk 的去噪推理天然可重叠: 预处理线程在 GPU 空闲间隙提交 H2D 与图像 kernel, 推理线程持续发射去噪 kernel。由于去噪是 launch-bound (GPU 留有 ~70% 空隙), 两者的 GPU 资源几乎不冲突。此项无需改动模型代码, 属部署编排。

4.4 FeatureTransform 向量化 (收尾项)

55 维 canonical 槽位的拼接/归一化目前是 Python 循环; 可预生成索引向量与 mean/std 常量, 用一次 index_select + 融合归一化替代。量级小 (ms 级), 在 4.1 完成后顺手处理。

5 预期收益与验收标准

表 3: 预期分解 (三相机 deploy 链路; 模型侧采用 dense MoE + compile 的当前值)

环节	现状 (ms)	目标 (ms)	手段
图像处理 (3 相机)	~950 (估)	30–80	§4.1
tokenizer + 模板	~5–15	~0 (episode 摊销)	§4.2
FeatureTransform	~20–50 (估)	<5	§4.4
模型推理	391	280–390 (+vision/prefix 编译)	后续工作
端到端	1442	<500	—

验收标准: ①预处理输出与 HF processor 逐元素一致 (容差 bf16 重结合级); ② A100 空闲机全链路实测 e2e < 500 ms (三相机) 且 preprocess < 100 ms; ③ 25 步 @ 30 Hz 预算 (833 ms) 满足; ④动作输出与现管线 parity 相对差 < 1%。

6 风险与兼容性

- **resize 数值对齐:**HF smart_resize 含取整规则 (28 的倍数网格), 双线性采样的边界行为需与 PIL/numpy 实现对齐——以逐元素测试锁定, 差异超差时回退 CPU 路径;
- **lerobot 处理器契约:**改动封装在 lingbot_vla_v2 自己的 processor 内,PolicyProcessorPipeline 框架不动;device_processor 提前后需保证后续步骤对 CUDA tensor 兼容;
- **回退路径:** 无 GPU / 调试场景保留现有 CPU 管线, 以开关切换。

7 里程碑

表 4: 实施顺序 (每项完成后即在 A100 出前后对比)

阶段	内容	预期 preprocess
M1	tokenizer 缓存 + FeatureTransform 向量化 (小项先行)	-20~-60 ms
M2	图像管线 GPU 化 + 逐元素等价测试	<100 ms
M3	全链路 A/B 实测 (空闲机)+ parity 报告	e2e <500 ms
M4	(可选) 跨 chunk 流水编排 + vision/prefix 纳入 compile	e2e ~430 ms

测量方法与数据出处

- [1] 官方 deploy 实测:accel-bench/deploy_compile_now.json(bf16+compile,robotwin 三相机,20 次迭代均值)。
- [2] lerobot 侧分段计时:bench/bench_lingbot_v2.py(CUDA event 探针),dense MoE 数据见姊妹篇《稠密双 GEMM MoE 技术报告》。
- [3] 对照基线: 官方 README 的 130 ms @4090D 声明仅覆盖 sample_actions, 不含预处理;A100 上该路径 model-only 实测 391.2 ms。